

Propuesta POO para Unity

Un ejemplo pequeno, original y practico: un Ghost DJ que lanza beats para activar objetivos danzantes.

Objetivo

Diseñar una estructura POO antes de programar ayuda a repartir responsabilidades: cada clase representa una cosa concreta del juego y sus relaciones quedan claras desde el principio.

Idea del ejemplo

En lugar de un inventario, el jugador controla un GhostDJ. Tiene equipado un BeatSpell, que a su vez sabe cual DanceTarget debe activar. La gracia esta en que los objetos se relacionan entre si sin convertirse en una clase gigante.

1. Las 3 clases

Clase	Responsabilidad	Atributos
GhostDJ	Representa al personaje y usa el beat equipado.	selectedBeat (privado), energy (publico), combo (privado)
BeatSpell	Representa el ataque/ritmo que afecta a un objetivo.	target (privado), power (publico), cooldown (privado)
DanceTarget	Representa el objeto que recibe beats y reacciona.	requiredHits (publico), danceTime (publico), hits (privado)

2. Relaciones entre objetos

La referencia principal es GhostDJ -> BeatSpell -> DanceTarget. Esto es parecido al concepto de 'objeto equipado', pero no es un inventario: el personaje conoce directamente la herramienta que utiliza, y la herramienta conoce a su objetivo actual.

Relacion	Que significa en Unity
GhostDJ tiene un BeatSpell	Una variable de tipo BeatSpell almacena la referencia al objeto del beat.
BeatSpell apunta a un DanceTarget	Otra variable de tipo DanceTarget identifica el objetivo que sera activado.
private + SerializeField	Mantiene el dato privado en codigo, pero editable desde el Inspector.

3. Propuesta de codigo

Para un proyecto real, la combinacion private + [SerializeField] suele ser muy util: el Inspector puede editar el valor sin hacer publica la variable para todo el codigo.

GhostDJ.cs

```
using UnityEngine;

public class GhostDJ : MonoBehaviour
{
    [SerializeField] private BeatSpell selectedBeat;
    public float energy = 100f;
    private int combo;

    public void PerformBeat()
    {
        if (selectedBeat != null && energy >= 10f)
        {
            selectedBeat.Cast();
            energy -= 10f;
            combo++;
        }
    }
}
```

BeatSpell.cs

```
using UnityEngine;

public class BeatSpell : MonoBehaviour
{
    [SerializeField] private DanceTarget target;
    public int power = 1;
    private float cooldown;

    public void Cast()
    {
        if (cooldown > 0f || target == null) return;
        target.ReceiveBeat(power);
        cooldown = 0.5f;
    }

    private void Update()
    {
        cooldown = Mathf.Max(0f, cooldown - Time.deltaTime);
    }
}
```

DanceTarget.cs

```
using UnityEngine;

public class DanceTarget : MonoBehaviour
{
    public int requiredHits = 3;
    public float danceTime = 2f;
    private int hits;

    public void ReceiveBeat(int power)
    {
        hits += power;
        if (hits >= requiredHits)
        {
            // Aqui podrias reproducir una animacion o efecto.
            hits = 0;
        }
    }
}
```

4. Como montarlo en la escena

Crea tres GameObjects: GhostDJ, BeatSpell y DanceTarget. Añade cada script a su objeto. En el Inspector de GhostDJ arrastra BeatSpell a Selected Beat. En el Inspector de BeatSpell arrastra DanceTarget a Target.

Luego puedes llamar a PerformBeat() desde un boton, una accion del Input System o cualquier otro sistema. El punto didactico es que cada clase hace una cosa distinta y las referencias expresan relaciones entre objetos.

Regla practica

Si una clase empieza a saber demasiado sobre movimiento, vida, audio, UI, enemigos y misiones a la vez, probablemente conviene separar responsabilidades en mas clases. En este ejercicio, el limite de 3 clases se mantiene a proposito para practicar la idea base.

5. Que es publico y que es privado

Tipo	Uso recomendado	Ejemplo
public	Dato o metodo que otras clases pueden necesitar directamente.	public float energy;
private	Estado interno que no deberia modificarse libremente desde fuera.	private int combo;
[SerializeField] private	Privado en codigo, editable desde el Inspector.	[SerializeField] private BeatSpell selectedBeat;